Assignment 3: LangGraph RAG Agent Report

CE8014 AI 代理系統之設計與開發

2026-04-08

Section 1: Implementation Logic

1.1 Architecture Overview

本專案實作一個 RAG (Retrieval-Augmented Generation) 系統，比較兩種 Agent 架構：

1. **GRAPH (LangGraph)** – 基於狀態機的 Agent，包含 Router、Grader、Generator、Rewriter 四個節點
2. **LEGACY (LangChain ReAct)** – 傳統的 ReAct Agent，使用 AgentExecutor 實現 Thought-Action-Observation 迴圈

系統架構分為四個檔案：

- **config.py** –LLM provider 配置，支援 Google、OpenAI-compatible、Anthropic
- **build_rag.py** –向量資料庫建立腳本，將 PDF 轉換為 ChromaDB
- **langgraph_agent.py** –GRAPH 與 LEGACY 兩種 Agent 實作
- **evaluator.py** –14 題測試用例評估腳本

1.2 LLM Model Configuration

本專案使用 **google/gemma-4-31b-it** 作為主要評估模型，透過 OpenRouter API 呼叫。

Gemma 4-31b-it 選擇原因：

Gemma 4 是 Google 最新的開源模型系列，相比上一代 Gemma 2 有顯著提升：

- **更強的推理能力**：在複雜任務如數學、編程、財務分析上表現優異
- **更大的上下文窗口**：支援 128K tokens，適合處理長文件
- **多語言支援**：財報問題包含中英混合，Gemma 4 能正確處理

配置方式：

```python
# config.py
def get_llm(temperature=0):
    provider = os.getenv("LLM_PROVIDER", "openai")
    return ChatOpenAI(
        model=os.getenv("OPENAI_MODEL", "google/gemma-4-31b-it"),
        base_url=os.getenv("OPENAI_BASE_URL", "https://openrouter.ai/api/v1"),
        temperature=temperature,
    )
```

1.3 GRAPH Agent 設計

LangGraph 採用狀態機架構，定義四個核心節點：

Node 1: retrieve_node (Router + Retrieve)

功能：根據問題內容路由到正確的資料來源。

```python
def retrieve_node(state: AgentState):
    question = state["question"]
    llm = get_llm()

    # Router: 判斷問題應該查詢 apple、tesla、both 或 none
    router_prompt = f"""You are a financial document router.
    Classify the user's question into exactly ONE of these categories:
    {json.dumps(["apple", "tesla", "both", "none"])}

    Rules:
    - If the question mentions only Apple -> "apple"
    - If the question mentions only Tesla -> "tesla"
    - If comparing both -> "both"
    - If unrelated -> "none"

    Output ONLY valid JSON: {{"datasource": "<category>"}}
    """

    response = llm.invoke(router_prompt)
    target = parse_json(response.content)

    # 根據路由結果檢索對應向量庫
    docs = retrievers[target].invoke(question)
    return {"documents": docs, "search_count": state["search_count"] + 1}
```

Node 2: grade_documents_node (Relevance Grader)

功能：評估檢索到的文件是否與問題相關。

```python
def grade_documents_node(state: AgentState):
    question = state["question"]
    documents = state["documents"]

    system_prompt = """You are a relevance grader.
    Determine whether the documents contain information that can help answer the

    IMPORTANT:
    - Look for specific financial figures, terms, or facts
    - If documents contain relevant data -> 'yes'
    - If documents are completely unrelated -> 'no'

    Answer with ONLY one word: 'yes' or 'no'.
    """

    response = llm.invoke([system_prompt, human_message])
```

```
    grade = "yes" if "yes" in response.content.lower() else "no"

    return {"needs_rewrite": grade}
```

Node 3: rewrite_node (Query Rewriter)

功能：當檢索失敗時，優化查詢語句。

```
def rewrite_node(state: AgentState):
    question = state["question"]

    msg = [
        SystemMessage(content="""You are a financial query optimizer.
        Rewrite vague questions into precise financial terminology.

        Examples:
        - 'how much did they spend on new tech' -> 'Research and Development (R&
        - 'how much money did they make' -> 'Total net sales / Total revenue'
        """),
        HumanMessage(content=f"Rewrite: '{question}'")
    ]

    response = llm.invoke(msg)
    return {"question": response.content.strip()}
```

Node 4: generate_node (Final Generator)

功能：根據檢索內容生成最終答案。

```
def generate_node(state: AgentState):
    question = state["question"]
    documents = state["documents"]

    prompt = ChatPromptTemplate.from_messages([
        ("system", """You are a financial analyst assistant.
        Answer based ONLY on the provided context.

        Rules:
        1. Answer in English
        2. Pay attention to fiscal years (2024 vs 2023 vs 2022)
        3. ALWAYS cite the source: [Source: Apple 10-K] or [Source: Tesla 10-K]
        4. If information not available, say "I don't know"
        5. Include exact numbers from the document

        Context: {context}"""),
        ("human", "{question}")
```

```
    ])

    chain = prompt | llm
    response = chain.invoke({"context": documents, "question": question})
    return {"generation": response.content}
```

State Graph Flow

```
START -> retrieve -> grade_documents -> decision
                                          |
                     yes ------------------------> generate -> END
                     |
                    no (and search_count <= 2) --> rewrite -> retrieve
                     |
                     no (and search_count > 2) --> generate -> END
```

1.4 LEGACY Agent 設計

LEGACY Agent 使用 LangChain 的 ReAct 模式：

```
def run_legacy_agent(question: str):
    tools = [
        create_retriever_tool(retriever, "search_apple_financials", "Apple 10-K'
        create_retriever_tool(retriever, "search_tesla_financials", "Tesla 10-K'
    ]

    template = """You are a senior financial analyst with access to SEC 10-K fil

    IMPORTANT RULES:
    1. Final Answer MUST be in English
    2. Pay attention to fiscal years
    3. If figure NOT found, say "I don't know"
    4. For comparison, search BOTH companies before answering
    5. Include specific numbers from documents

    Use the following format:
    Thought: <reasoning>
    Action: <tool_name>
    Action Input: <query>
    Observation: <result>
    ... (repeat)
    Thought: I now know the final answer
    Final Answer: <answer>
    """
```

```
    agent = create_react_agent(llm, tools, prompt)
    agent_executor = AgentExecutor(agent=agent, tools=tools, max_iterations=5)

    return agent_executor.invoke({"input": question})["output"]
```

1.5 Embedding Model 選擇

實驗比較兩種 Embedding Model：

Model	維度	語言	MTEB 排名
sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2	384	多語言	中
BAAI/bge-base-en-v1.5	**768**	**英文專用**	**高**

BGE 在英文檢索任務上表現優異，適合財報文件（全英文）。

Section 2: Experiment Results

2.1 Test Case 修正

作業原始 test case 有 4 處 Tesla 數字錯誤，經驗證後修正：

Test	原始值	正確值	來源頁碼
B (R&D)	4.77 billion	**$4,540 million**	Tesla 10-K p.52
E (Energy)	23.7 billion	**$10,086 million**	Tesla 10-K p.52
A2 (Auto)	78,512 million	**$72,480 million**	Tesla 10-K p.52 (2024)
B2 (CapEx)	11,153 million	**$11,339 million**	Tesla 10-K p.55

驗證方法：直接查閱 Tesla 2024 10-K 財報 PDF + NotebookLM 交叉驗證。

2.2 實驗配置

Version	Embedding Model	Chunk Size	Chunk Overlap
V2	MiniLM (多語言)	1000	200
V3	MiniLM (多語言)	2000	400
V4	**BGE (英文)**	**2000**	**400**

2.3 結果總覽

Version	GRAPH	LEGACY	Delta
V2	11/14 (78.6%)	9/14 (64.3%)	+2
V3	12/14 (85.7%)	8/14 (57.1%)	+4
V4	**14/14 (100%)**	7/14 (50.0%)	**+7**

2.4 V4 詳細結果 (BGE + chunk2000 + GRAPH)

```
Test A: Apple Revenue - PASS (18.56s)
  Answer: Apple's total net sales for 2024 was $391,035 million.

Test B: Tesla R&D - PASS (36.40s)
  Answer: Tesla's R&D expenses for 2024 were $4,540 million.

Test D: Apple Services Cost - PASS (32.19s)
  Answer: Apple's cost of sales for services was $25,119 million.

Test E: Tesla Energy Revenue - PASS (81.88s)
  Answer: Tesla's Energy generation and storage revenue was $10,086 million.

Test G: Unknown Info - PASS (46.54s)
  Answer: The documents do not contain iPhone 17 pricing information.

Test A1: Apple Revenue [Eng] - PASS (15.38s)
Test A2: Tesla Automotive [Eng] - PASS (8.62s)
Test B1: Apple R&D [Mixed] - PASS (5.21s)

Test B2: Tesla CapEx [Mixed] - PASS (5.57s)
  Answer: Tesla's capital expenditures were $11,339 million.

Test C1: R&D Comparison [Eng] - PASS (30.71s)
  Answer: Apple: $31,370 million; Tesla: $4,540 million.
          Apple spent more.

Test C2: Gross Margin [Eng] - PASS (99.25s)
  Answer: Apple: ~46.2%; Tesla: 17.9%. Apple higher.

Test D1: Apple Services Cost [Eng] - PASS (18.20s)
Test E1: 2025 Projection [Mixed] - PASS (38.85s)
Test F1: CEO Identity [Eng] - PASS (5.91s)

FINAL SCORE: 14/14 (100%)
```

Section 3: Analysis

3.1 LangGraph vs LangChain 比較

維度	GRAPH (LangGraph)	LEGACY (LangChain)
架構	狀態機	ReAct 迴圈
錯誤恢復	Query rewriting 機制	依賴模型自我修正
平均分數	**12.3/14**	8/14
穩定性	高	低
可解釋性	每個節點職責清晰	迴圈行為難以追蹤

GRAPH 優勢：

1. **狀態管理**：AgentState 明確追蹤 search_count、needs_rewrite 等狀態
2. **條件分支**：grade_documents 節點可根據相關性決定是否重寫查詢
3. **重試上限**：search_count > 2 時強制生成，避免無限迴圈

LEGACY 劣勢：

1. **缺乏狀態追蹤**：無法知道已經嘗試過哪些查詢
2. **依賴模型能力**：如果模型無法自我修正，就會陷入死胡同
3. **格式解析脆弱**：Minimax M2.7 輸出格式不符合 ReAct 規範導致解析失敗

3.2 Embedding Model 影響

Embedding	GRAPH (chunk2000)	特點
MiniLM (多語言)	12/14	比較題失敗 (C1/C2)
BGE (英文)	**14/14**	全部通過

BGE 優勢：

1. **英文專用**：財報為英文文件，BGE 針對英文優化
2. **更高維度**：768 vs 384，語義表達更豐富
3. **MTEB 排名**：BGE 在檢索任務上排名較高

MiniLM 問題：

比較題（C1/C2）需要同時檢索 Apple 和 Tesla 數據，MiniLM 的語義匹配能力不足，導致檢索到的 Tesla 數據不完整。

3.3 Chunk Size Trade-off

Chunk Size	Tesla Chunks	Test B2 (CapEx)	Notes
1000	~500+	FAIL	表格碎片化，失去上下文
2000	**338**	**PASS**	完整表格，檢索成功

分析：

- Tesla 10-K 有 144 頁，包含大量財務表格
- Chunk size 1000 會將表格切碎，單個 chunk 缺乏完整上下文
- Chunk size 2000 能保留完整表格結構

Trade-off：

- **優點**：更大的 chunk 保留更多上下文
- **缺點**：可能引入雜訊，LEGACY 模式反而下降（9 分降至 8 分）

Section 4: Conclusion

最佳配置

```
Embedding Model: BAAI/bge-base-en-v1.5
Chunk Size: 2000
Chunk Overlap: 400
Agent Mode: GRAPH (LangGraph)
LLM: google/gemma-4-31b-it
Score: 14/14 (100%)
```

關鍵發現

1. **GRAPH 優於 LEGACY**：LangGraph 的狀態機架構提供更好的錯誤恢復和流程控制，平均領先 LEGACY 4-7 分。
2. **Embedding 選擇關鍵**：英文財報應使用英文專用的 BGE embedding，比多語言 MiniLM 提升 2 分。
3. **Chunk Size 影響檢索**：大文件（Tesla 144 頁）需要更大的 chunk size (2000) 來保留表格完整性。
4. **Test Case 驗證重要**：作業原始 test case 有 4 處數字錯誤，需先驗證才能正確評估系統性能。

未來改進方向

1. **Grader 優化**：當前 grader 有時會誤判相關文件為不相關，可調整 prompt 或使用更強模型
2. **Router 改進**：部分查詢被錯誤路由到 “none”，可增加 domain-specific 關鍵詞
3. **LEGACY 格式兼容**：針對不同模型調整 ReAct prompt 格式